



Ad Soyad: _____

Numara: _____

Tarih: _____

1 Çalışma 1: Görsel Yükleme ve Gösterme

Örnek Kod

Aşağıdaki programı `gorsel_test.py` dosyasına yazın ve çalıştırın:

```
1 import os
2 import pygame
3
4 pygame.init()
5
6 GENISLIK = 800
7 YUKSEKLIK = 600
8
9 ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
10 pygame.display.set_caption("Gorsel Yükleme Testi")
11 saat = pygame.time.Clock()
12
13 # Dosya dizini
14 DIZIN = os.path.dirname(os.path.abspath(__file__))
15 GORSEL = os.path.join(DIZIN, "assets", "images")
16
17 # Gorsel yukle
18 dosya_yolu = os.path.join(GORSEL, "playerShip1_blue.png")
19 try:
20     gemi = pygame.image.load(dosya_yolu).convert_alpha()
21     print("[OK] Gorsel yuklendi")
22 except FileNotFoundError:
23     print("[UYARI] Dosya bulunamadi, placeholder olusturuluyor")
24     gemi = pygame.Surface((64, 64), pygame.SRCALPHA)
25     gemi.fill((255, 0, 255, 180))
26
27 calistir = True
28 while calistir:
29     for olay in pygame.event.get():
30         if olay.type == pygame.QUIT:
31             calistir = False
32
33     ekran.fill((10, 10, 40))
34     ekran.blit(gemi, (GENISLIK // 2 - 32, YUKSEKLIK // 2 - 32))
35     pygame.display.flip()
36     saat.tick(60)
37
38 pygame.quit()
```

Görsel Dosyası Hazırlığı

kenney.nl/assets adresinden “Space Shooter Redux” paketini indirin. Proje klasörünüzde `assets/images/` dizini oluşturup görselleri oraya kopyalayın. Eğer indirmediyse program placeholder görsel oluşturacaktır.

Talep 1: Kendi Görselinizi Yükleyin

Farklı bir görsel dosyası (PNG veya JPG) yükleyin ve ekranın sol üst köşesine (50, 50) konumuna çizin. Terminaldeki [OK] mesajını gözlemleyin.

Talep 2: Birden Fazla Görsel

En az 3 farklı görsel yükleyin (gemi, düşman, arka plan) ve her birini ekranın farklı konumlarına çizin. Çizim sırasını değiştirerek z-order etkisini gözlemleyin.

Talep 3: gorsel_yukle() Fonksiyonu

Var olmayan bir dosya adı vererek `FileNotFoundError` hatasını test edin. Ardından kitaptaki `gorsel_yukle()` fonksiyonunu yazın: dosya bulunamazsa magenta renkli ve çapraz çizgili bir placeholder döndürsün.

2 Çalışma 2: convert() ve convert_alpha() Karşılaştırması

Örnek Kod

Aşağıdaki kodu `convert_test.py` dosyasına yazın:

```
1 import os
2 import time
3 import pygame
4
5 pygame.init()
6 ekran = pygame.display.set_mode((800, 600))
7
8 DIZIN = os.path.dirname(os.path.abspath(__file__))
9 dosya = os.path.join(DIZIN, "assets", "images", "playerShip1_blue.png")
10
11 # 3 farkli yukleme yontemi
12 try:
13     ham = pygame.image.load(dosya) # Donusumsuz
14     donusmus = pygame.image.load(dosya).convert() # convert
15     alpha = pygame.image.load(dosya).convert_alpha() # convert_alpha
16 except FileNotFoundError:
17     print("Gorsel bulunamadi, cikiliyor.")
18     pygame.quit()
19     exit()
20
21 # Performans testi
22 def blit_testi(gorsel, tekrar=10000):
23     baslangic = time.time()
24     for _ in range(tekar):
25         ekran.blit(gorsel, (0, 0))
26     return (time.time() - baslangic) * 1000
27
28 print(f"Ham : {blit_testi(ham):.2f} ms")
29 print(f"convert : {blit_testi(donusmus):.2f} ms")
30 print(f"convert_a: {blit_testi(alpha):.2f} ms")
31
32 pygame.quit()
```

Talep 1: Performans Ölçümü

Kodu çalıştırın ve 3 yöntemin süresini aşağıdaki tabloya yazın:

Ham (dönüşümsüz)	
<code>convert()</code>	
<code>convert_alpha()</code>	

Talep 2: Görsel Farkı Gözlemleme

Dama deseni arkaplan üzerine hem `convert()` ile hem `convert_alpha()` ile yüklenmiş görseli yan yana çizin. Alpha kanalının korunup korunmadığını gözlemleyerek hangi yöntemin şeffaflığı koruduğunu belirleyin.

Talep 3: `set_colorkey()` Deneyi

`convert()` ile yüklenmiş bir görseli `set_colorkey((0, 0, 0))` uygulayın. Siyah piksellerin şeffaf olup olmadığını gözlemleyin. Ardından farklı renklerle (beyaz, kırmızı) deneyin ve sonuçları kaydedin.

3 Çalışma 3: Katmanlı Sahne Oluşturma

Örnek Kod

Aşağıdaki kodu `katmanli_sahne.py` dosyasına yazın:

```
1 import os
2 import random
3 import pygame
4
5 pygame.init()
6 GENISLIK, YUKSEKLIK = 800, 600
7 ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
8 saat = pygame.time.Clock()
9
10 DIZIN = os.path.dirname(os.path.abspath(__file__))
11
12 def gorsel_yukle(ad, boyut=None):
13     yol = os.path.join(DIZIN, "assets", "images", ad)
14     try:
15         g = pygame.image.load(yol).convert_alpha()
16         if boyut:
17             g = pygame.transform.scale(g, boyut)
18         return g
19     except (FileNotFoundError, pygame.error):
20         yt = pygame.Surface(boyut or (64, 64), pygame.SRCALPHA)
21         yt.fill((255, 0, 255, 180))
22         return yt
23
24 # Yildizli arkaplan olustur
25 arkaplan = pygame.Surface((GENISLIK, YUKSEKLIK))
26 arkaplan.fill((5, 5, 25))
27 for _ in range(150):
28     x = random.randint(0, GENISLIK)
29     y = random.randint(0, YUKSEKLIK)
```

```

30     arkaplan.set_at((x, y), (200, 200, 200))
31
32 # Gorseller
33 gemi = gorsel_yukle("playerShip1_blue.png", (64, 64))
34 dusman = gorsel_yukle("enemyRed1.png", (48, 48))
35
36 # 4 dusman konumu
37 dusmanlar = [(100 + i * 160, 100) for i in range(4)]
38
39 çalıştır = True
40 while çalıştır:
41     for olay in pygame.event.get():
42         if olay.type == pygame.QUIT:
43             çalıştır = False
44
45     fare_x, fare_y = pygame.mouse.get_pos()
46
47     # Z-ORDER: sıra önemli!
48     ekran.blit(arkaplan, (0, 0))          # 1. Arkaplan
49     for dx, dy in dusmanlar:              # 2. Dusmanlar
50         ekran.blit(dusman, (dx, dy))
51     ekran.blit(gemi, (fare_x - 32, fare_y - 32)) # 3. Oyuncu
52
53     pygame.display.flip()
54     saat.tick(60)
55
56 pygame.quit()

```

Talep 1: Sahneyi Test Edin

Kodu çalıştırın. Fareyi hareket ettirerek oyuncu gemisinin düşmanların üzerinden geçtiğini doğrulayın. Çizim sırasını değiştirerek (gemiye düşmanlardan önce çizim) ne olduğunu gözlemleyin.

Talep 2: Düşman Hareketi

Düşman gemilerini yatay olarak sağa doğru hareket ettirin. Ekranın sağ kenarından çıkan düşmanı soldan tekrar sokun (wrap stratejisi).

Talep 3: UI Katmanı Ekleyin

Ekranın sol üst köşesine bir skor metni (`pygame.font`) ve sağ üst köşesine FPS bilgisi ekleyin. Bu yazıları tüm görsellerin **üstünde** çizdiğinizden emin olun (z-order).

4 Çalışma 4: Boyut Değiştirme

Örnek Kod

Aşağıdaki programı `boyut_demo.py` dosyasına yazın ve çalıştırın:

```

1 import pygame
2
3 pygame.init()
4 GENISLIK = 800

```

```

5 YUKSEKLIK = 600
6
7 ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
8 pygame.display.set_caption("Boyut Degistirme")
9 saat = pygame.time.Clock()
10
11 # Yedek gorsel olustur (64x64 kare)
12 orijinal = pygame.Surface((64, 64), pygame.SRCALPHA)
13 pygame.draw.polygon(orijinal, (0, 150, 255),
14                     [(32, 4), (4, 60), (60, 60)])
15
16 # Farkli boyutlarda kopyalar
17 kucuk = pygame.transform.scale(orijinal, (32, 32))
18 buyuk = pygame.transform.scale(orijinal, (128, 128))
19 smooth = pygame.transform.smoothscale(orijinal, (128, 128))
20
21 calistir = True
22 while calistir:
23     for olay in pygame.event.get():
24         if olay.type == pygame.QUIT:
25             calistir = False
26
27     ekran.fill((20, 20, 40))
28
29     ekran.blit(kucuk, (100, 250))
30     ekran.blit(orijinal, (250, 220))
31     ekran.blit(buyuk, (400, 190))
32     ekran.blit(smooth, (600, 190))
33
34     pygame.display.flip()
35     saat.tick(60)
36
37 pygame.quit()

```

scale() ve smoothscale() Farkı

`scale()` hızlı ama pikselleşme olabilir. `smoothscale()` daha kaliteli ama daha yavaştır. Farkı 128x128 boyutundaki iki görselde karşılaştırın.

Talep 1: Boyut Etiketleri

Her görselin altına boyutunu gösteren metin ekleyin. `pygame.font.SysFont("Arial", 16)` ile yazı tipi oluşturun. Örneğin kucuk görselin altına "32x32" yazın.

Talep 2: En-Boy Oranı Koruma

Aşağıdaki fonksiyonu programınıza ekleyin ve dikdörtgen bir görsel (100x50) oluşturup en-boy oranını koruyarak genişliği 200'e ölçeklendirin. Sonucu oranı bozulmuş bir versiyonla (200x200) yan yana gösterin.

```
1 def boyut_oranli(surface, hedef_genislik):
2     oran = hedef_genislik / surface.get_width()
3     yeni_yuk = int(surface.get_height() * oran)
4     return pygame.transform.scale(
5         surface, (hedef_genislik, yeni_yuk))
```

Talep 3: Dinamik Boyutlandırma

Fare tekerleğiyle (MOUSEWHEEL olayı) görselin boyutunu artırıp azaltın. `olay.y` yukarı kaydırmada +1, aşağıda -1 döner. Minimum 16, maksimum 256 piksel sınırı koyun. Her boyut değişikliğinde **orijinalden** ölçeklendirme yapın.

5 Çalışma 5: Döndürme ve Çevirme

Örnek Kod – Döndürme

Aşağıdaki programı `dondurma_demo.py` dosyasına yazın:

```
1 import pygame
2
3 pygame.init()
4 ekran = pygame.display.set_mode((600, 400))
5 pygame.display.set_caption("Dondurma Demo")
6 saat = pygame.time.Clock()
7
8 # Ucgen seklinde gorsel
9 orijinal = pygame.Surface((64, 64), pygame.SRCALPHA)
10 pygame.draw.polygon(orijinal, (255, 200, 0),
11                     [(32, 4), (4, 60), (60, 60)])
12 aci = 0
13
14 çalıştır = True
15 while çalıştır:
16     for olay in pygame.event.get():
17         if olay.type == pygame.QUIT:
18             çalıştır = False
19
20     aci += 2
21     if aci >= 360:
22         aci -= 360
23
24     # Orijinalden dondur, center koru
25     donmus = pygame.transform.rotate(orijinal, aci)
26     rect = donmus.get_rect(center=(300, 200))
27
28     ekran.fill((20, 20, 40))
29     ekran.blit(donmus, rect)
30     pygame.display.flip()
```

```
31     saat.tick(60)
32
33 pygame.quit()
```

Birikimli Döndürme Hatası

Orijinali saklamadan döndürme yapmak (her karede önceki sonucu döndürmek) kalite kaybına ve sürekli büyüyen bir Surface'e neden olur. Her zaman **orijinalden** döndürün.

Talep 1: Döndürme Hızı Kontrolü

Yukarı/aşağı ok tuşlarıyla döndürme hızını artırıp azaltın. `hiz` değişkeni oluşturun (başlangıç: 2). Yukarı ok ile artırın, aşağı ok ile azaltın. Hız değerini ekranda gösterin.

Talep 2: Fareye Döndürme

`math.atan2()` kullanarak görseli farenin konumuna doğru döndürün. Gemi ile fare arasına ince bir çizgi çizin. Açığı ekranda gösterin.

```
1 import math
2 dx = fare_x - nesne_x
3 dy = fare_y - nesne_y
4 aci = math.degrees(math.atan2(-dy, dx)) - 90
```

Talep 3: Karakter Yön Değiştirme

`cevirme_demo.py` dosyası oluşturun. Sağa bakan bir üçgen çizin. `pygame.transform.flip()` ile sol yönlü versiyonunu oluşturun. Ok tuşlarıyla hareket ederken yöne göre doğru görseli gösterin. Karakterin altına yarı saydam bir yansıması ekleyin:

```
1 yansima = pygame.transform.flip(gorsel, False, True)
2 yansima.set_alpha(80)
3 ekran.blit(yansima, (x, y + gorsel.get_height()))
```

6 Çalışma 6: Sprite Sınıfı ve Gruplar

Örnek Kod – Temel Sprite

Aşağıdaki programı `temel_sprite.py` dosyasına yazın ve çalıştırın:

```
1 import pygame
2 import sys
3
4 pygame.init()
5
6 GENISLIK = 800
7 YUKSEKLIK = 600
8
9 ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
10 pygame.display.set_caption("Temel Sprite")
11 saat = pygame.time.Clock()
```

```

12
13 # Basit bir Sprite sınıfı
14 class Kutu(pygame.sprite.Sprite):
15     def __init__(self, renk, x, y, boyut=50):
16         super().__init__()
17         self.image = pygame.Surface((boyut, boyut), pygame.SRCALPHA)
18         pygame.draw.rect(self.image, renk,
19                          (0, 0, boyut, boyut), border_radius=5)
20         self.rect = self.image.get_rect(center=(x, y))
21
22     def update(self):
23         pass # Simdilik bos
24
25 # Sprite oluşturun
26 kutu = Kutu((0, 180, 255), GENISLIK // 2, YUKSEKLIK // 2)
27 grup = pygame.sprite.Group(kutu)
28
29 çalıştır = True
30 while çalıştır:
31     saat.tick(60)
32     for olay in pygame.event.get():
33         if olay.type == pygame.QUIT:
34             çalıştır = False
35
36     grup.update()
37     ekran.fill((20, 20, 30))
38     grup.draw(ekran)
39     pygame.display.flip()
40
41 pygame.quit()
42 sys.exit()

```

Sprite ve Group

Her **Sprite** alt sınıfı **image** (görünüm) ve **rect** (konum) özelliklerine sahip olmalıdır. **Group** bu özellikleri kullanarak **draw()** ve **update()** işlemlerini toplu yapar.

Talep 1: Birden Fazla Sprite ve Hareket

Üç farklı renkte ve konumda kutu oluşturun. Hepsini aynı gruba ekleyin. **update()** metoduna ok tuşlarıyla hareket ekleme kodu yazın. Kutunun WASD veya ok tuşlarıyla dört yönde hareket etmesini sağlayın. **rect.clamp_ip()** ile ekran sınırları içinde tutun.

Talep 2: Grup İşlemleri

Aşağıdaki kodu yazın ve test edin:

```
1 import random
2
3 class Top(pygame.sprite.Sprite):
4     def __init__(self):
5         super().__init__()
6         self.renk = (random.randint(100, 255),
7                     random.randint(100, 255),
8                     random.randint(100, 255))
9         self.image = pygame.Surface((20, 20), pygame.SRCALPHA)
10        pygame.draw.circle(self.image, self.renk, (10, 10), 10)
11        self.rect = self.image.get_rect(
12            center=(random.randint(50, 750),
13                  random.randint(50, 550)))
14        self.hiz_x = random.choice([-3, -2, 2, 3])
15        self.hiz_y = random.choice([-3, -2, 2, 3])
16
17    def update(self):
18        self.rect.x += self.hiz_x
19        self.rect.y += self.hiz_y
20        if self.rect.left < 0 or self.rect.right > 800:
21            self.hiz_x *= -1
22        if self.rect.top < 0 or self.rect.bottom > 600:
23            self.hiz_y *= -1
```

15 adet `Top` oluşturup gruba ekleyin. Space ile yeni top ekleyin, R ile rastgele bir topu `kill()` ile silin. Eleman sayısını ekranda gösterin.

Talep 3: Kayan Yıldız Arka Planı

Aşağıdaki `Yildiz` sınıfını kullanarak 50 yıldızlık bir arka plan oluşturun:

```
1 class Yildiz(pygame.sprite.Sprite):
2     def __init__(self):
3         super().__init__()
4         self.boyut = random.randint(1, 3)
5         cap = self.boyut * 2
6         self.image = pygame.Surface((cap, cap), pygame.SRCALPHA)
7         parlaklik = 100 + self.boyut * 50
8         renk = (parlaklik, parlaklik, parlaklik)
9         pygame.draw.circle(self.image, renk,
10                           (self.boyut, self.boyut), self.boyut)
11        self.rect = self.image.get_rect()
12        self.rect.x = random.randint(0, 800)
13        self.rect.y = random.randint(0, 600)
14        self.hiz = self.boyut * 0.8 + random.uniform(0, 1)
15
16    def update(self):
17        self.rect.y += self.hiz
18        if self.rect.top > 600:
19            self.rect.x = random.randint(0, 800)
20            self.rect.bottom = 0
```

Yıldızları `tum_spritelar` ve `yildizlar` gruplarına ekleyin. Paralaks derinlik efekti ekleyin: boyut 1 ya-
vaş (uzak), boyut 3 hızlı (yakın).

7 Çalışma 7: Mini Proje – Uzay Sahnesi

Proje Gereksinimleri

Aşağıdaki gereksinimleri karşılayan bir **uzay sahnesi** programı yazın:

- Ok tuşları veya WASD ile kontrol edilen bir **Oyuncu** Sprite'ı
- En az 60 adet kayan **Yıldız** arka planı
- **tum_spritelar** ve **yildizlar** olmak üzere en az 2 grup
- **clamp_ip()** ile ekran sınırı kontrolü
- Sol üst köşede hayatta kalma süresi sayacı (saniye)
- Tüm güncelleme ve çizim işlemleri grup metodlarıyla yapılmalı

Talep 1: Temel Yapı

Yukarıdaki gereksinimleri karşılayan programı yazın. Oyuncu gemisi ekranın alt ortasında başlamalıdır.

Talep 2: FPS ve HUD

Sağ üst köşeye FPS değerini, sol üst köşeye hayatta kalma süresini gösteren bir bilgi paneli ekleyin. **saat.get_fps()** fonksiyonunu kullanın.

Talep 3: Egzoz Efektı

Geminin arkasından küçük turuncu/sarı daireler bırakan bir egzoz efekti ekleyin. Her karede geminin altında yeni bir **Egzoz** Sprite'ı oluşturun. Egzoz Sprite'ları birkaç kare sonra **kill()** ile kendini silsin.

Bonus: Meteor Engellerinden Kaçış

Ekranın üstünden rastgele konumlarda aşağı doğru kayan **Meteor** Sprite'ları ekleyin. Meteorlar farklı boyut ve hızlarda olsun. Henüz çarpışma algılama yapmıyoruz ama meteorların görsel olarak tehdit oluşturmalarını sağlayın. Meteor grubunu ayrı tutun.

- Her 2 saniyede bir yeni meteor
- 3 farklı boyut (küçük, orta, büyük)
- Büyük meteorlar daha yavaş, küçükler hızlı
- Ekran dışına çıkan meteorlar **kill()** ile silinsin

Başarılar!